In [9]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [10]:
```
# Load the dataset (change this path as per your local directory!)
file_path = 'california_housing_train (1).csv'
data = pd.read_csv(file_path)
```

In [11]:
```
# Display the first few rows
data.head()
```

Out[11]:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population
0	-114.31	34.19	15.0	5612.0	1283.0	1015.0
1	-114.47	34.40	19.0	7650.0	1901.0	1129.0
2	-114.56	33.69	17.0	720.0	174.0	333.0
3	-114.57	33.64	14.0	1501.0	337.0	515.0
4	-114.57	33.57	20.0	1454.0	326.0	624.0

In [12]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17000 entries, 0 to 16999
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   longitude           17000 non-null  float64
 1   latitude            17000 non-null  float64
 2   housing_median_age  17000 non-null  float64
 3   total_rooms         17000 non-null  float64
 4   total_bedrooms      17000 non-null  float64
 5   population          17000 non-null  float64
 6   households          17000 non-null  float64
 7   median_income       17000 non-null  float64
 8   median_house_value  17000 non-null  float64
dtypes: float64(9)
memory usage: 1.2 MB
```

What do you understand from the above ?

In [13]:
```
data.describe()
```

Out[13]:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
count	17000.000000	17000.000000	17000.000000	17000.000000	17000.000000
mean	-119.562108	35.625225	28.589353	2643.664412	539.410824
std	2.005166	2.137340	12.586937	2179.947071	421.499452
min	-124.350000	32.540000	1.000000	2.000000	1.000000
25%	-121.790000	33.930000	18.000000	1462.000000	297.000000
50%	-118.490000	34.250000	29.000000	2127.000000	434.000000
75%	-118.000000	37.720000	37.000000	3151.250000	648.250000
max	-114.310000	41.950000	52.000000	37937.000000	6445.000000

Your interpreations ?

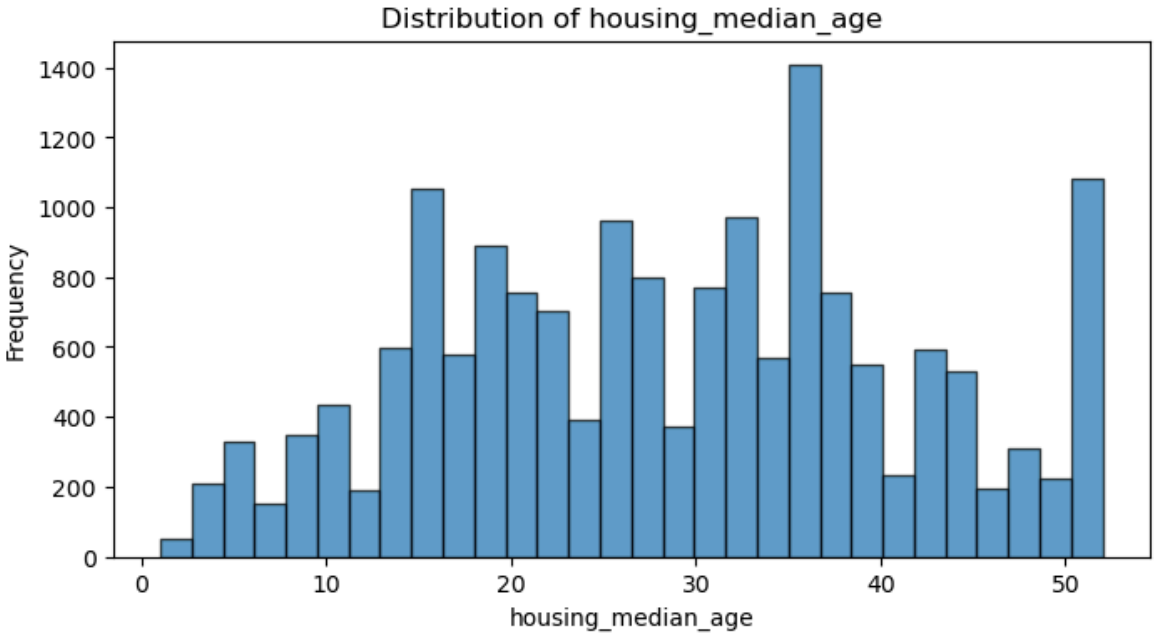
Maybe you can try visualising some of these aspects to make them more engaging, and create more room for discussion ?

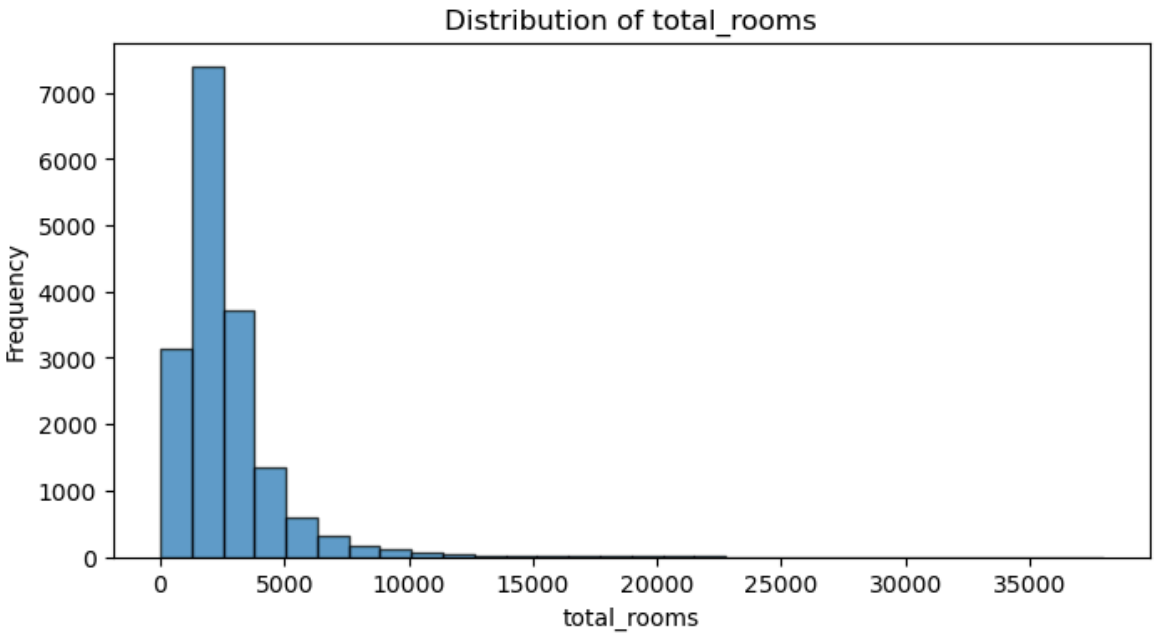
Visualize Distributions

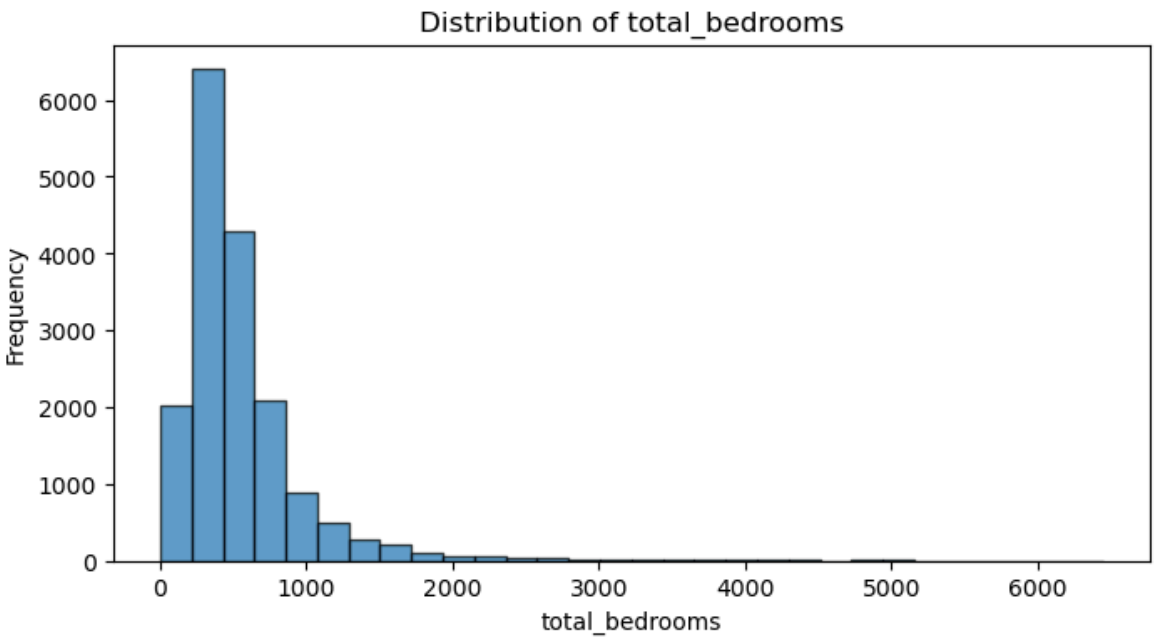
The histograms display the distributions of the key variables, showing patterns such as skewness or clustering. This helps us understand the nature of each variable in the dataset.

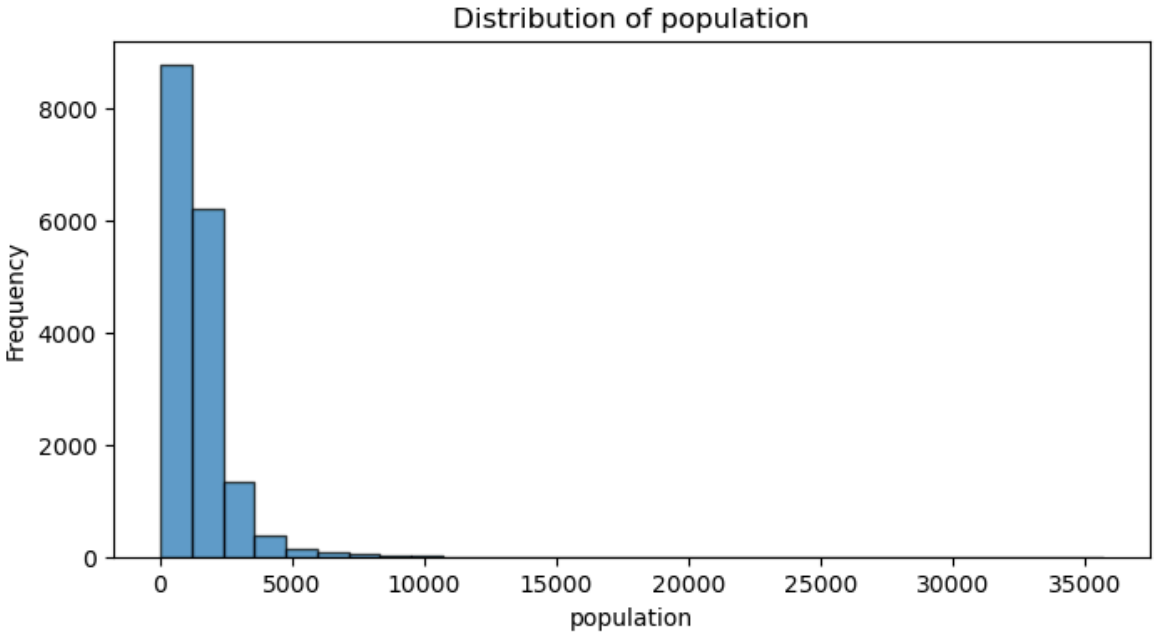
In [14]:

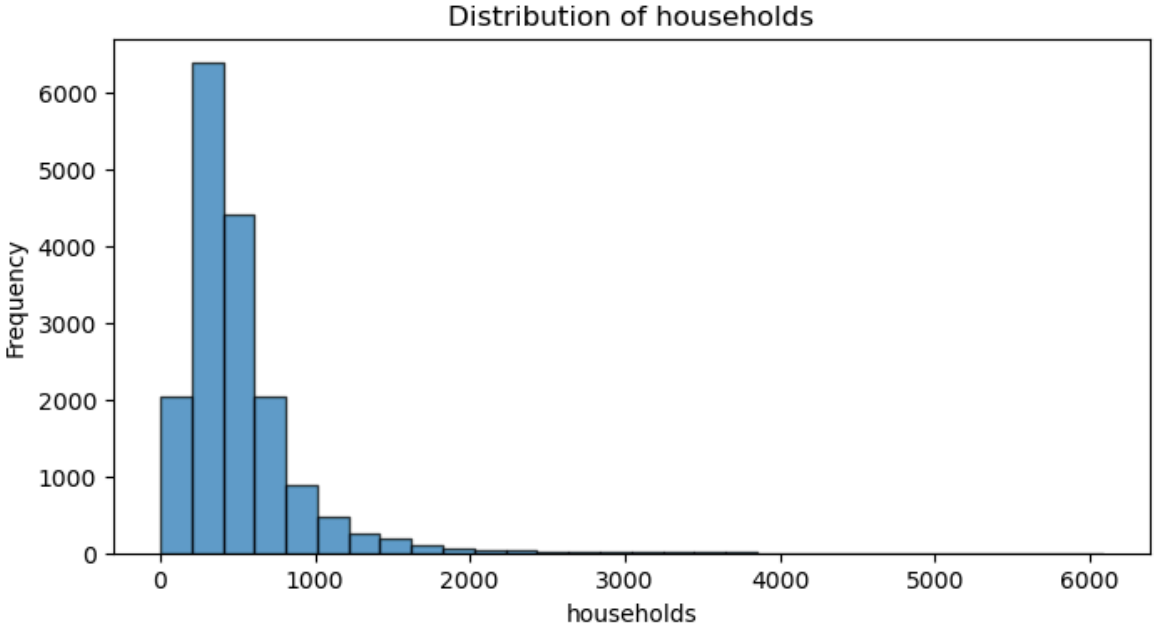
```
# Plotting distributions for key variables
columns_to_plot = ['housing_median_age', 'total_rooms', 'total_bedrooms',
                   'population', 'households', 'median_income', 'median_house_va

for column in columns_to_plot:
    plt.figure(figsize=(8, 4))
    plt.hist(data[column], bins=30, edgecolor='k', alpha=0.7)
    plt.title(f'Distribution of {column}')
    plt.xlabel(column)
    plt.ylabel('Frequency')
    plt.show()
    print("--" * 10)
```

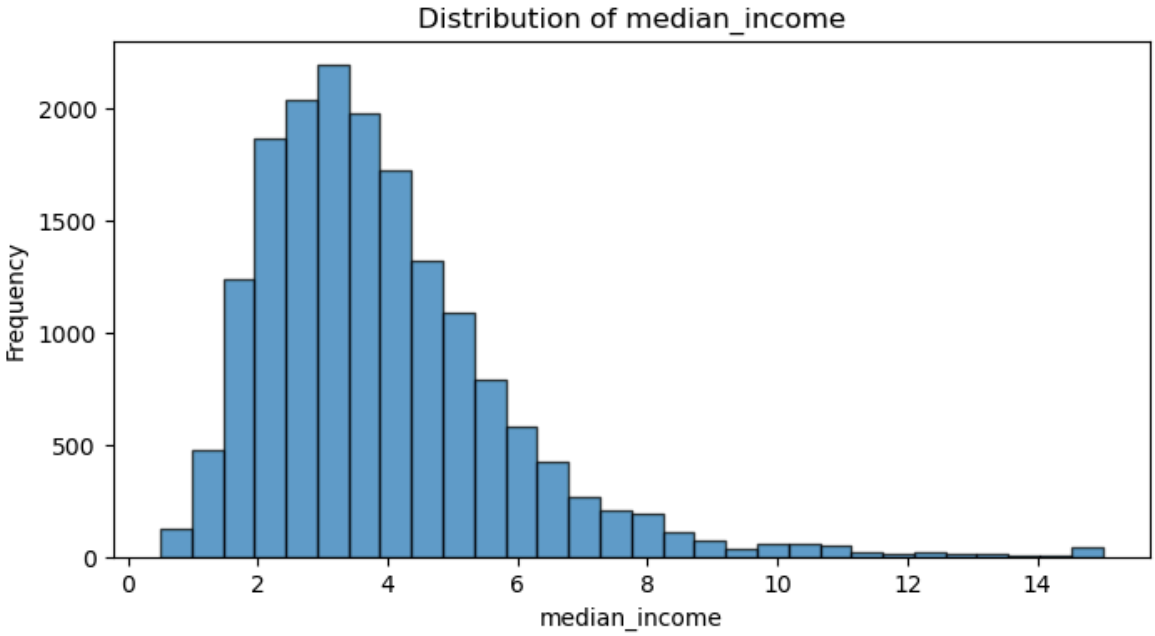

Distribution of housing_median_age
Frequency
housing_median_age
0
200
400
600
800
1000
1200
1400
0
10
20
30
40
50

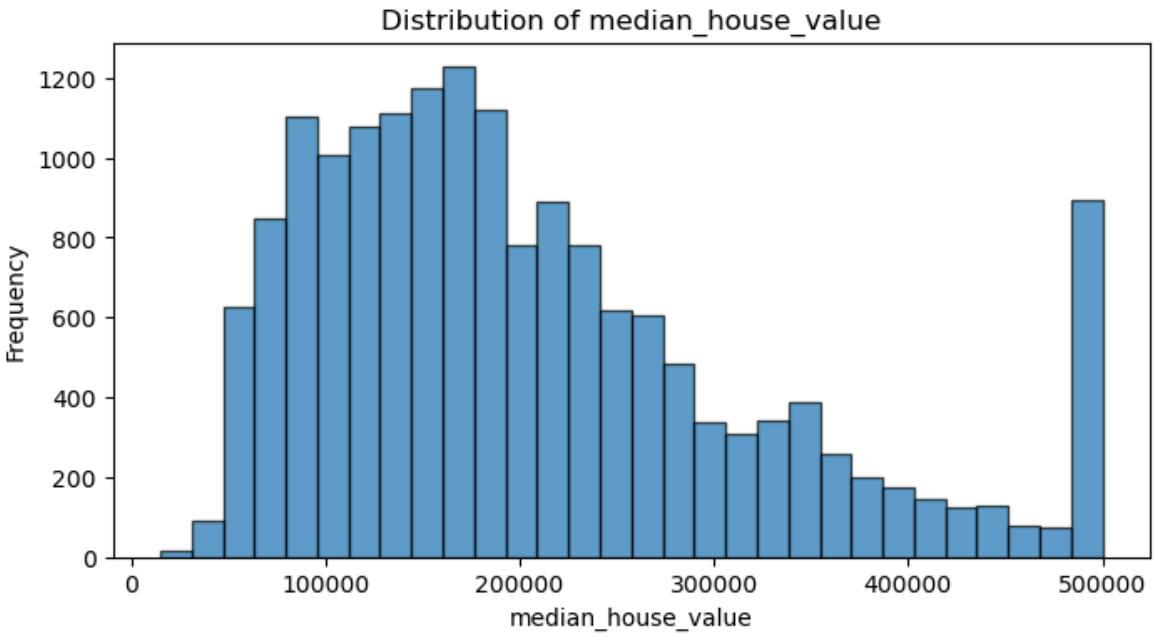

Distribution of total_rooms
Frequency
total_rooms
0
1000
2000
3000
4000
5000
6000
7000
0
5000
10000
15000
20000
25000
30000
35000


Distribution of total_bedrooms
Frequency
total_bedrooms
0
1000
2000
3000
4000
5000
6000
0
1000
2000
3000
4000
5000
6000


Distribution of population
Frequency
8000
6000
4000
2000
0
0
5000
10000
15000
20000
25000
30000
35000
population


Distribution of households
Frequency
6000
5000
4000
3000
2000
1000
0
0
1000
2000
3000
4000
5000
6000
households





Your interpreations ?

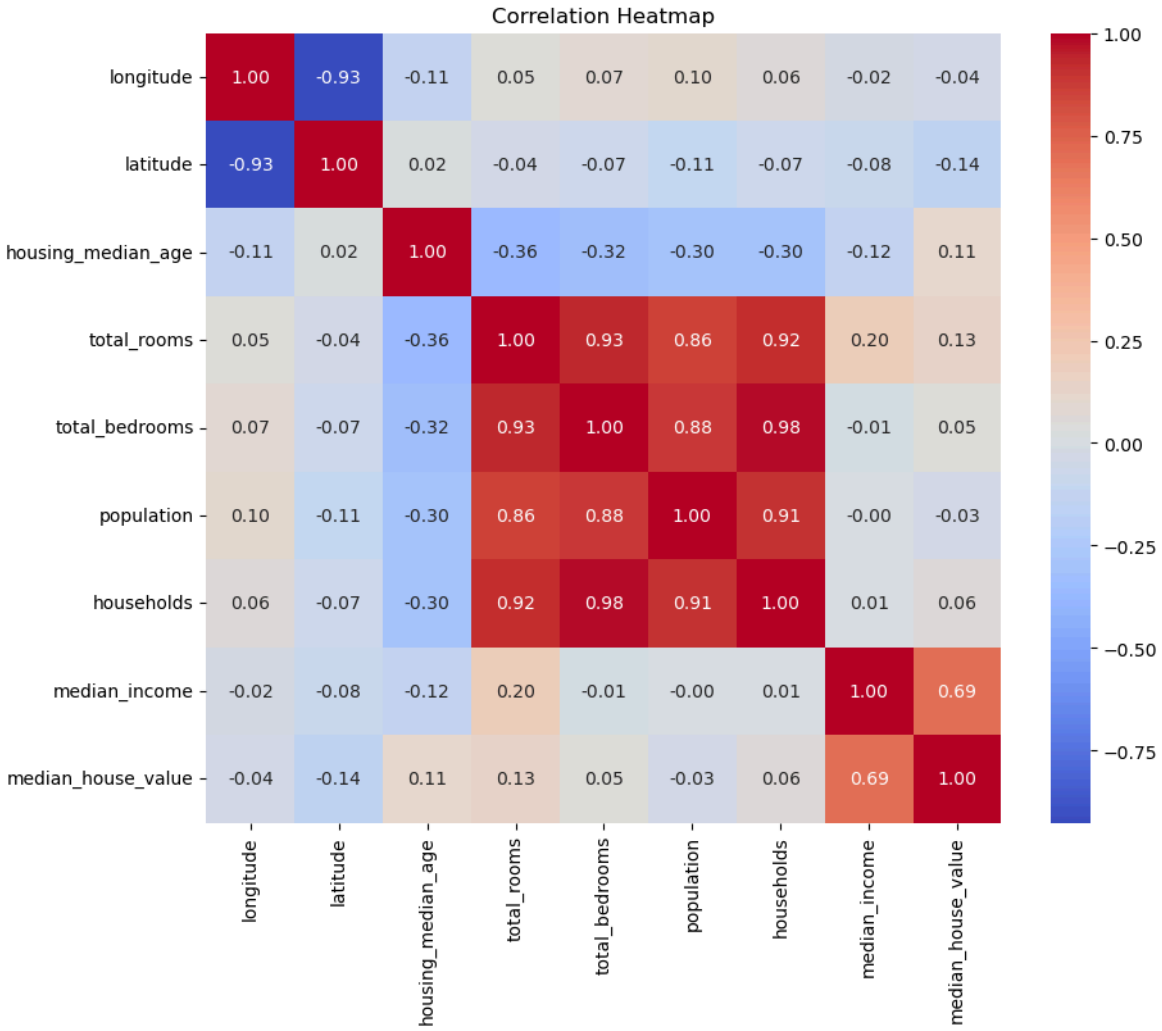
In [ ]:

```
```

Correlation

In [15]:

```
# Correlation heatmap
plt.figure(figsize=(10, 8))
correlation_matrix = data.corr()
sns.heatmap(correlation_matrix, annot=True, fmt='.2f', cmap='coolwarm', cbar=Tru
plt.title('Correlation Heatmap')
plt.show()
```

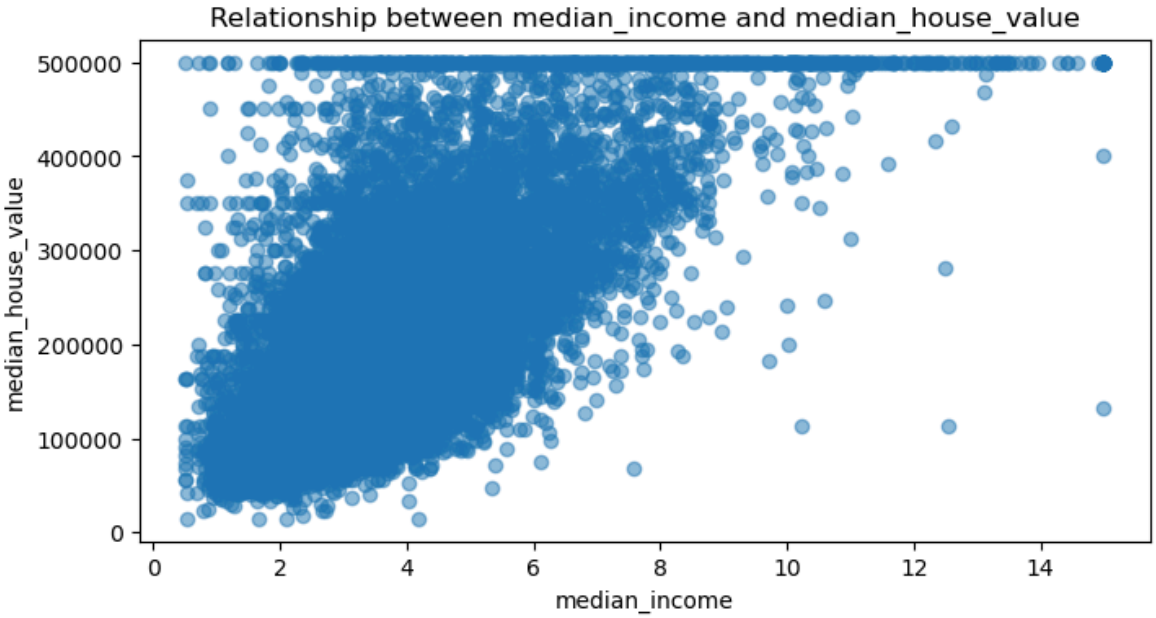


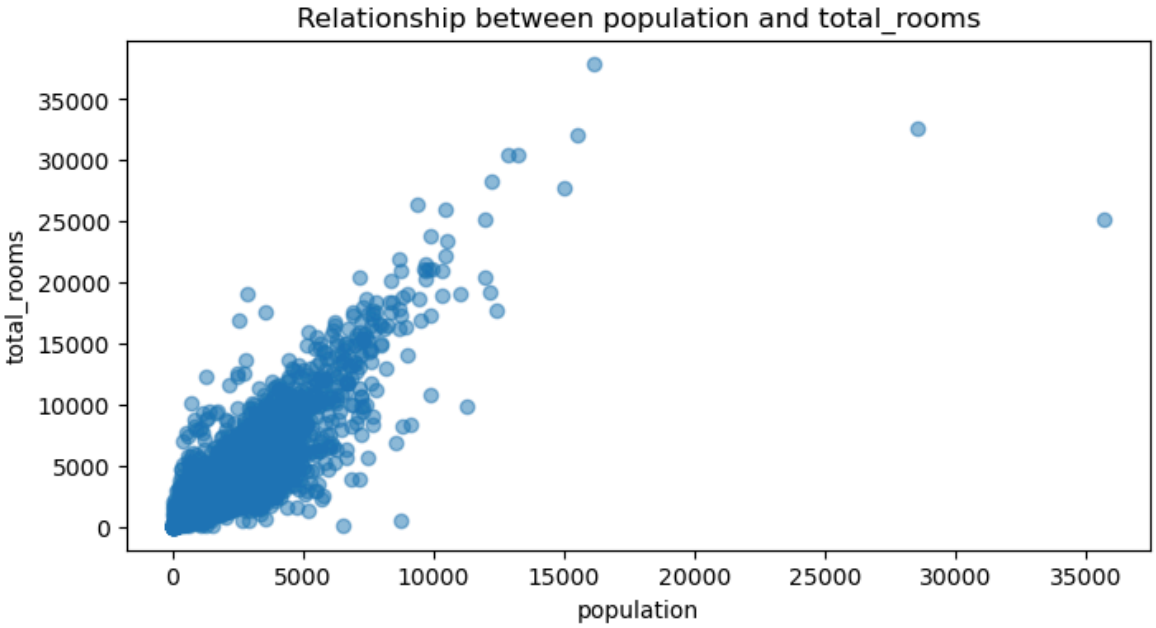
What do you think about above ? What does this reveal ? Did you learn something ? Are you more informed about dataset ?

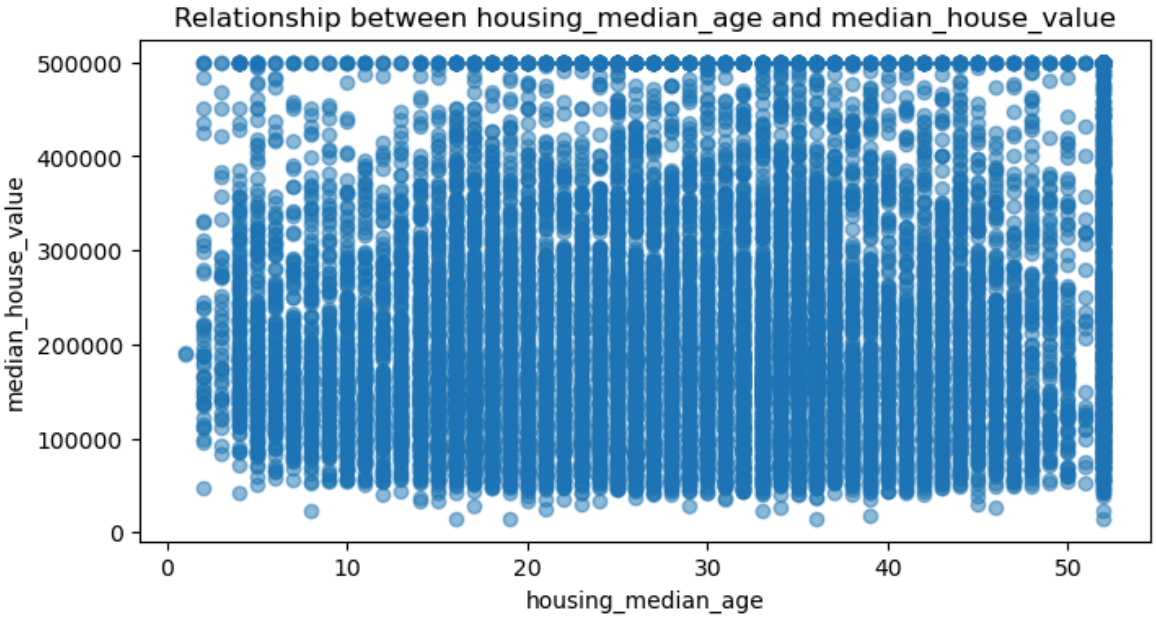
Explore Relationships

Now, let's explore relationships between variables, such as income vs. house value or population vs. total rooms, using scatter plots.

In [16]:
```
# Scatter plots to explore relationships
scatter_pairs = [
    ('median_income', 'median_house_value'),
    ('population', 'total_rooms'),
    ('housing_median_age', 'median_house_value')
]

for x, y in scatter_pairs:
    plt.figure(figsize=(8, 4))
    plt.scatter(data[x], data[y], alpha=0.5)
    plt.title(f'Relationship between {x} and {y}')
    plt.xlabel(x)
    plt.ylabel(y)
    plt.show()
    print("--" * 10)
```


Relationship between median_income and median_house_value
median_house_value
median_income
0
100000
200000
300000
400000
500000
0
2
4
6
8
10
12
14


Relationship between population and total_rooms
total_rooms
population
0
5000
10000
15000
20000
25000
30000
35000
0
5000
10000
15000
20000
25000
30000
35000

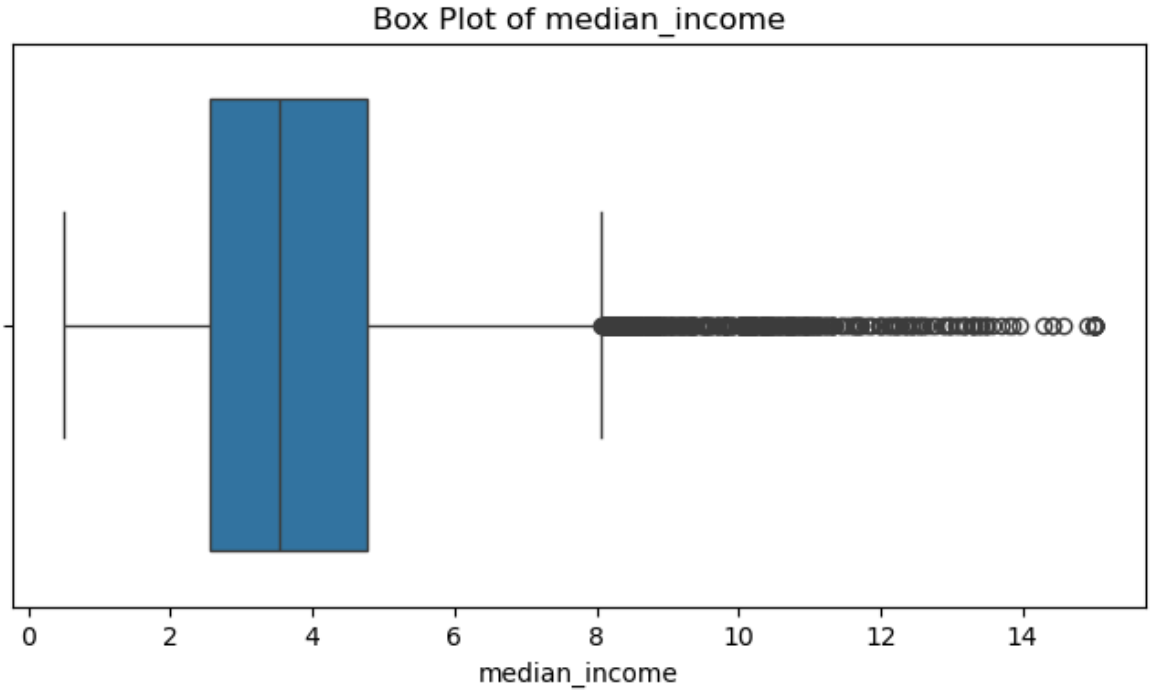

Relationship between housing_median_age and median_house_value
median_house_value
housing_median_age
0
100000
200000
300000
400000
500000
0
10
20
30
40
50

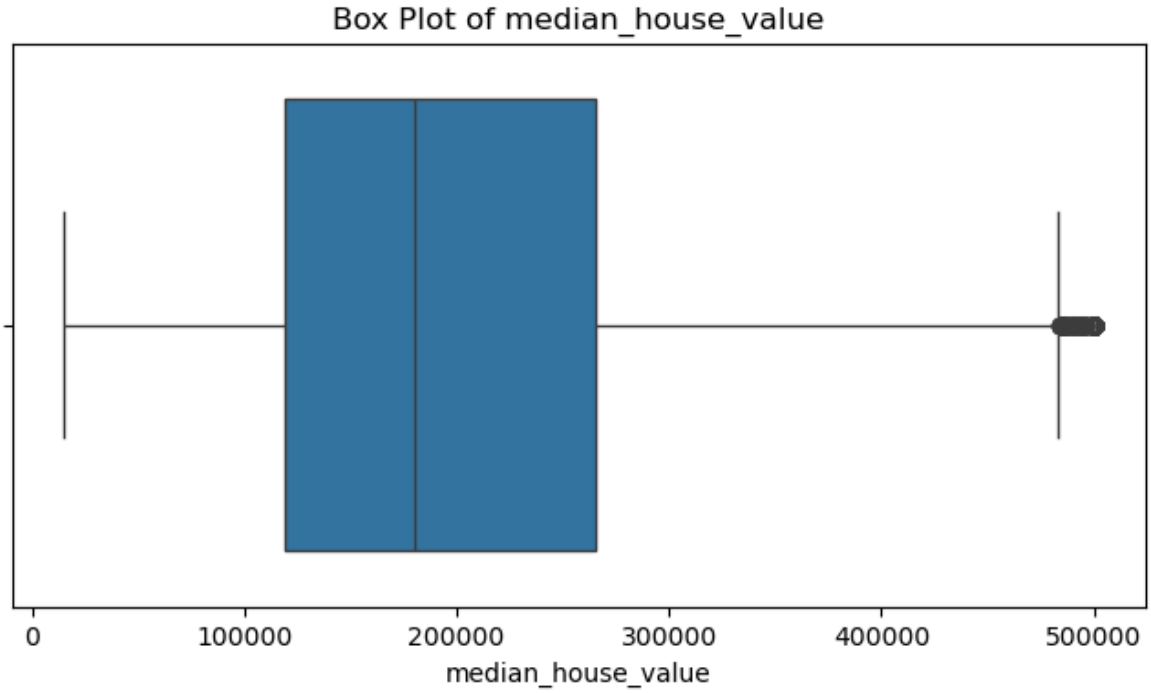
The scatter plots highlight the relationships between selected variable pairs:

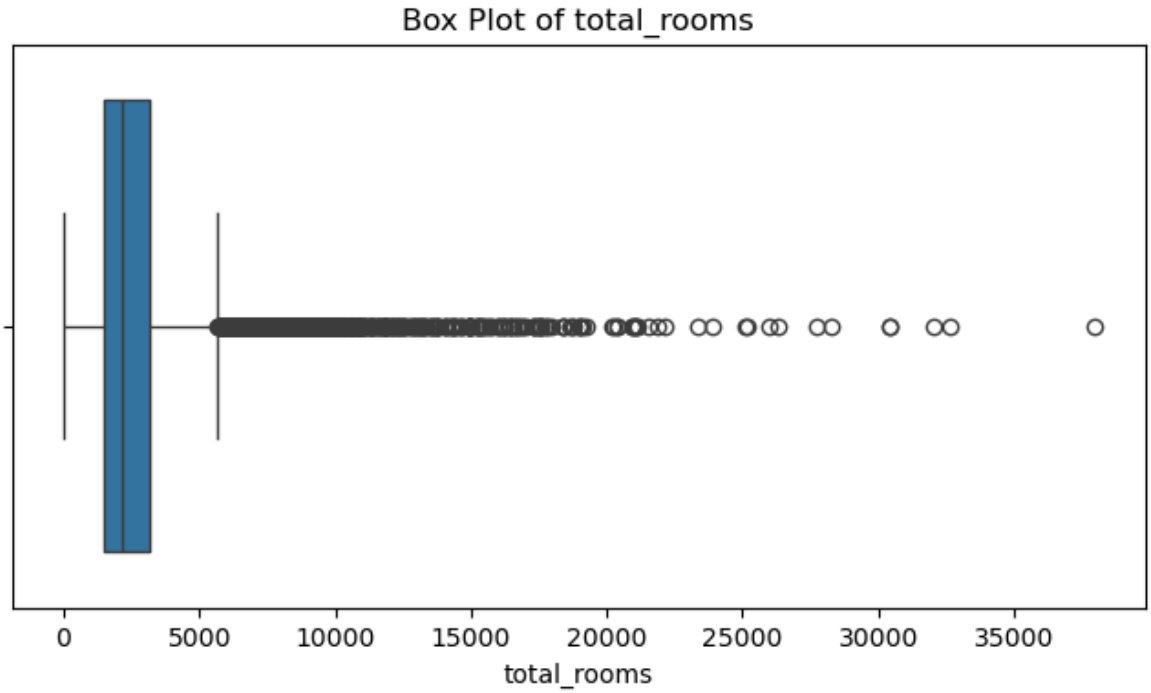
- Income vs. House Value: A clear positive correlation exists, with higher income often corresponding to higher house values.
- Population vs. Total Rooms: Shows a scattered trend indicating variability in housing density.
- Housing Age vs. House Value: Suggests a potential relationship worth exploring further.

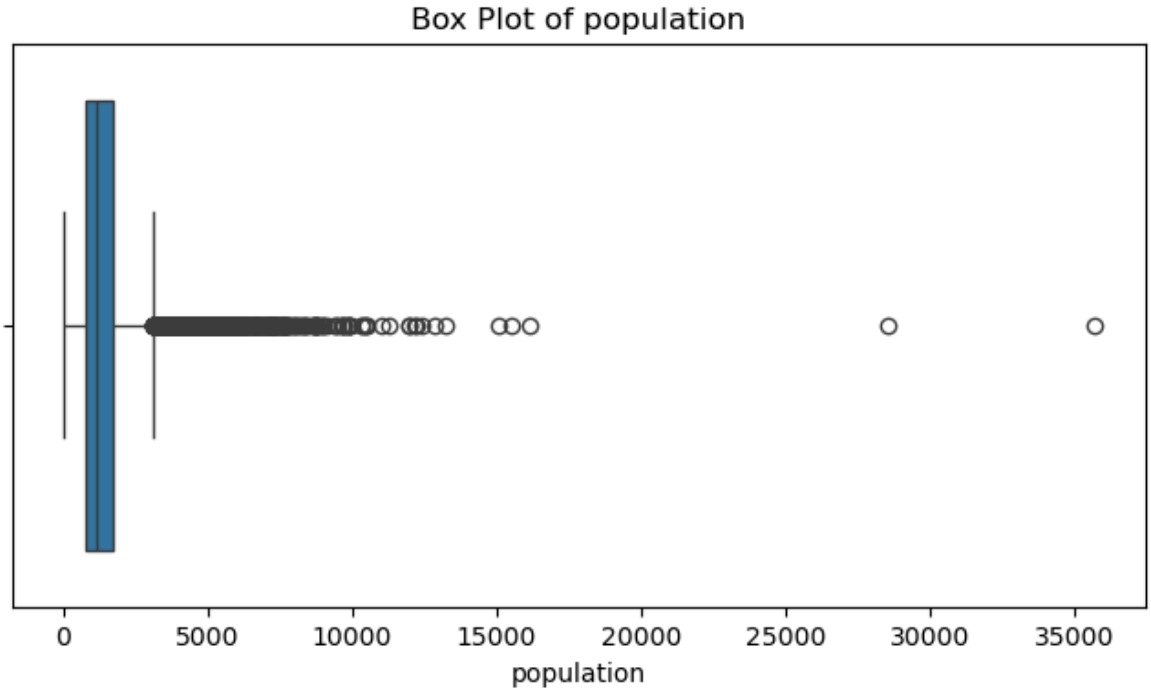
Boxplots

In [17]:
```
# Box plots for key numerical variables
for column in ['median_income', 'median_house_value', 'total_rooms', 'population
    plt.figure(figsize=(8, 4))
    sns.boxplot(data=data, x=column)
    plt.title(f'Box Plot of {column}')
    plt.xlabel(column)
    plt.show()
    print("--" * 10)
```




Box Plot of median_house_value
0
100000
200000
300000
400000
500000
median_house_value

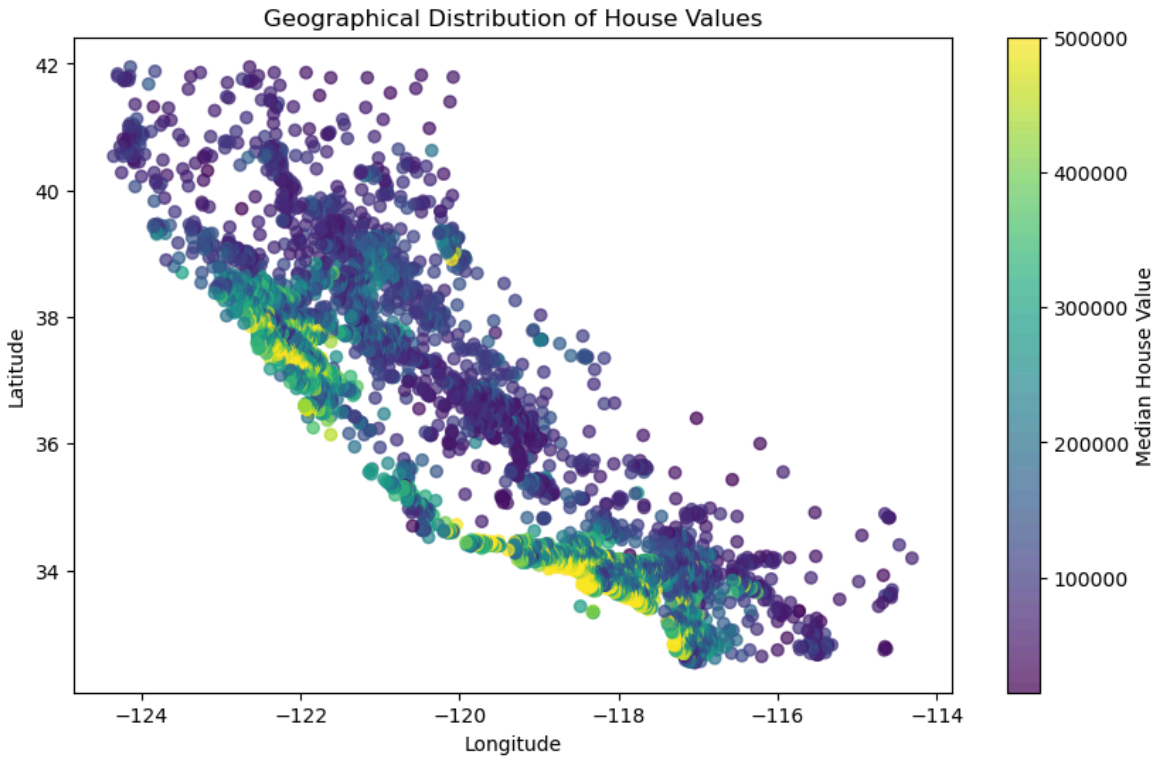

Box Plot of total_rooms
0
5000
10000
15000
20000
25000
30000
35000
total_rooms



Geographical Trends

Let's examine geographical trends using latitude and longitude to visualize the distribution of median house values.

In [18]:
```
# Scatter plot for geographical trends
plt.figure(figsize=(10, 6))
scatter = plt.scatter(data['longitude'], data['latitude'], c=data['median_house_
                      cmap='viridis', alpha=0.7)
plt.colorbar(scatter, label='Median House Value')
plt.title('Geographical Distribution of House Values')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.show()
```

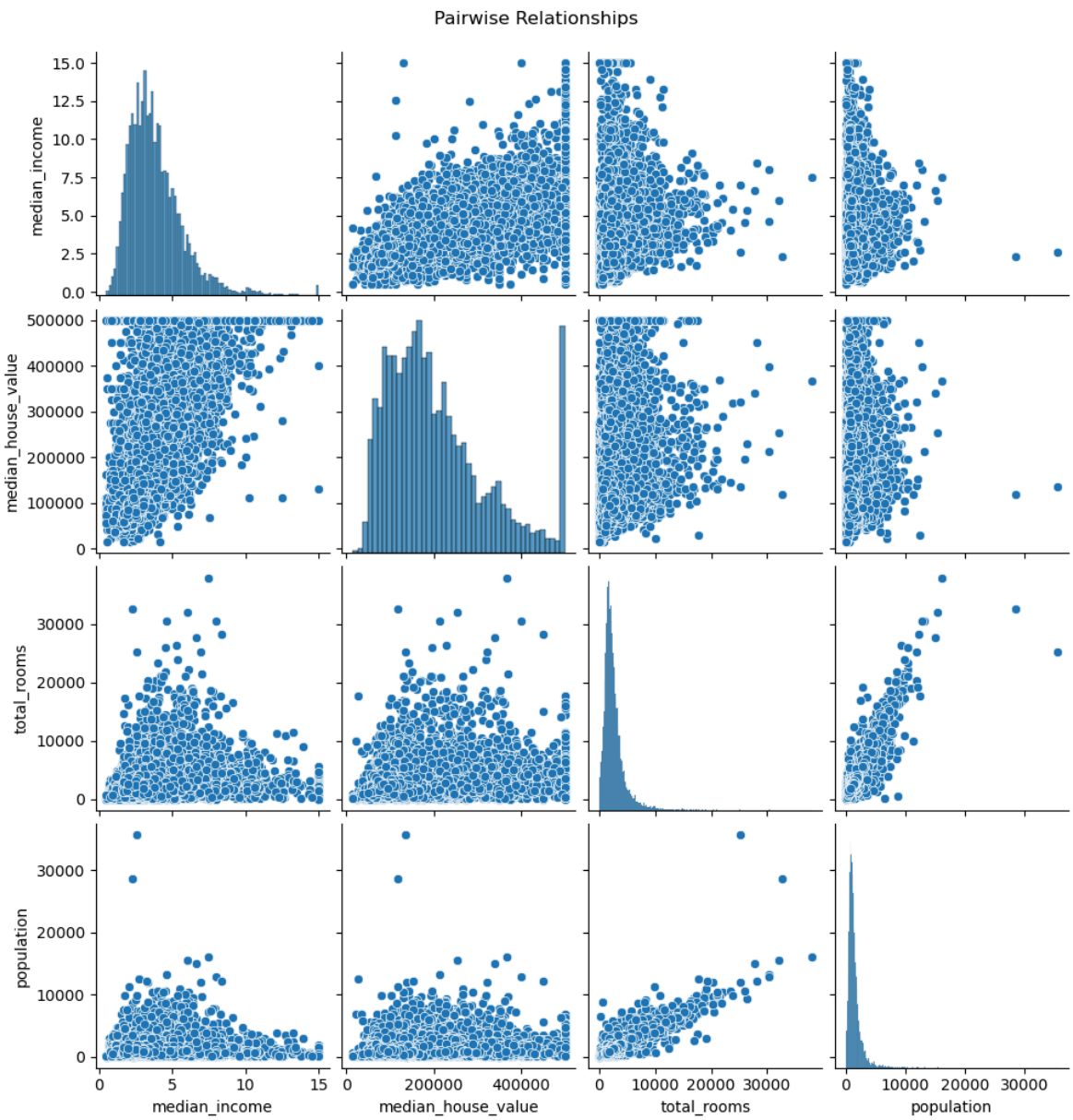


The geographical scatter plot reveals spatial trends in housing values, with clustering in certain regions. For example, areas with higher house values appear near specific coordinates, likely influenced by location factors like proximity to urban centers or coastlines.

Pairplot

Scatterplot matrix (pairplot) to observe pairwise relationships.

In [19]:
```
# Pairplot to visualize pairwise relationships
sns.pairplot(data[['median_income', 'median_house_value', 'total_rooms', 'popula
plt.suptitle('Pairwise Relationships', y=1.02)
plt.show()
```



In [ ]:

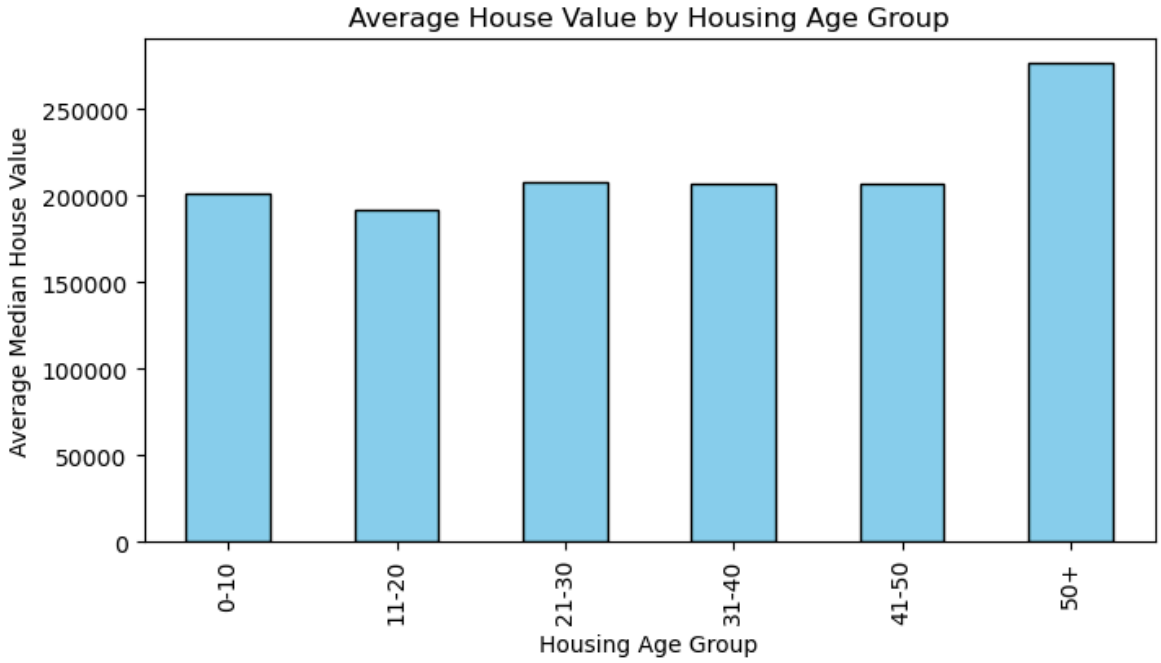
In [ ]:

Binned Analysis

Visualizing distributions using binning (e.g., by housing age or income range).

In [20]:
```
# Binned analysis for housing_median_age
binned_age = pd.cut(data['housing_median_age'], bins=[0, 10, 20, 30, 40, 50, 100
binned_data = data.groupby(binned_age)['median_house_value'].mean()

plt.figure(figsize=(8, 4))
binned_data.plot(kind='bar', color='skyblue', edgecolor='k')
plt.title('Average House Value by Housing Age Group')
plt.xlabel('Housing Age Group')
plt.ylabel('Average Median House Value')
plt.show()
```

```
C:\Users\S2120431\AppData\Local\Temp\ipykernel_27724\3874341148.py:3: FutureWarni
ng: The default of observed=False is deprecated and will be changed to True in a
future version of pandas. Pass observed=False to retain current behavior or obser
ved=True to adopt the future default and silence this warning.
  binned_data = data.groupby(binned_age)['median_house_value'].mean()
```



In [ ]:

In [ ]:

In [ ]:

In [ ]:

Data Cleaning

In [21]:
```
# Data Cleaning: Check for missing values
missing_values = data.isnull().sum()
print("Missing Values:\n", missing_values)
```

```
Missing Values:
 longitude             0
latitude              0
housing_median_age    0
total_rooms           0
total_bedrooms        0
population            0
households            0
median_income         0
median_house_value    0
dtype: int64
```

In [22]:
```
# This step might not make sense but is done to give an idea of how values can b
# Handling missing values by imputing with the median
data_cleaned = data.fillna(data.median())
print("Cleaned Data:\n", data_cleaned.head())
```

```
Cleaned Data:
    longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0     -114.31     34.19                15.0       5612.0          1283.0
1     -114.47     34.40                19.0       7650.0          1901.0
2     -114.56     33.69                17.0        720.0           174.0
3     -114.57     33.64                14.0       1501.0           337.0
4     -114.57     33.57                20.0       1454.0           326.0

   population  households  median_income  median_house_value
0      1015.0       472.0         1.4936             66900.0
1      1129.0       463.0         1.8200             80100.0
2       333.0       117.0         1.6509             85700.0
3       515.0       226.0         3.1917             73400.0
4       624.0       262.0         1.9250             65500.0
```

In [23]:
```
# Verifying that no missing values remain
missing_values_after_cleaning = data_cleaned.isnull().sum()
print("Missing Values After Cleaning:\n", missing_values_after_cleaning)
```

```
Missing Values After Cleaning:
 longitude             0
latitude              0
housing_median_age    0
total_rooms           0
total_bedrooms        0
population            0
households            0
median_income         0
median_house_value    0
dtype: int64
```

In [ ]:

In [ ]:

Feature Engineering

Feature Engineering to create additional useful features and standardize the dataset

In [24]:
```
# Feature Engineering: Create new features
data_cleaned['rooms_per_household'] = data_cleaned['total_rooms'] / data_cleaned
data_cleaned['bedrooms_per_room'] = data_cleaned['total_bedrooms'] / data_cleane
data_cleaned['population_per_household'] = data_cleaned['population'] / data_cle
```

In [25]:
```
# Standardizing the numerical features for better performance in ML models
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
numerical_columns = ['housing_median_age', 'total_rooms', 'total_bedrooms',
                     'population', 'households', 'median_income',
                     'rooms_per_household', 'bedrooms_per_room', 'population_per


data_cleaned[numerical_columns] = scaler.fit_transform(data_cleaned[numerical_co
```

Test Train Split

In [26]:
```
# Proceed to train-test split
from sklearn.model_selection import train_test_split

X = data_cleaned.drop(['median_house_value'], axis=1)
y = data_cleaned['median_house_value']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

In [27]:
```
X_train.shape, X_test.shape
```

Out[27]:
```
((13600, 11), (3400, 11))
```

In [ ]:

Machine Learning - Regression

Regression is used for numerical predictions. There are multiple type of regression models. We'll use only 2 at this instance to understand the process.

Both of the models (a systematical mathemtaical process or algorithm) will solve the same problem but they will approach it differently.

- (1st Model) LinearRegression
- (2nd Model) RandomForestRegressor

Model evaluation metrics:

- mean_squared_error, r2_score

In [28]:
```
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

In [29]:
```
# Train Linear Regression model
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
```

Out[29]:
```
▾ LinearRegression
LinearRegression()
```

In [30]:
```
# Train Random Forest model
rf_model = RandomForestRegressor(random_state=42)
rf_model.fit(X_train, y_train)
```

Out[30]:
```
▾ RandomForestRegressor
RandomForestRegressor(random_state=42)
```

In [31]:
```
# Predictions
lr_predictions = lr_model.predict(X_test)
rf_predictions = rf_model.predict(X_test)
```

In [32]:
```
# Evaluation metrics
lr_rmse = mean_squared_error(y_test, lr_predictions, squared=False)
lr_r2 = r2_score(y_test, lr_predictions)

rf_rmse = mean_squared_error(y_test, rf_predictions, squared=False)
rf_r2 = r2_score(y_test, rf_predictions)
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:492: Fu
tureWarning: 'squared' is deprecated in version 1.4 and will be removed in 1.6. T
o calculate the root mean squared error, use the function'root_mean_squared_erro
r'.
  warnings.warn(
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:492: Fu
tureWarning: 'squared' is deprecated in version 1.4 and will be removed in 1.6. T
o calculate the root mean squared error, use the function'root_mean_squared_erro
r'.
  warnings.warn(
```

In [33]:
```
# Displaying the results
evaluation_results = pd.DataFrame({
    "Model": ["Linear Regression", "Random Forest"],
    "RMSE": [lr_rmse, rf_rmse],
    "R² Score": [lr_r2, rf_r2]
})
```

In [34]:
```
evaluation_results
```

Out[34]:

	Model	RMSE	R^2 Score
0	Linear Regression	67471.942597	0.669605
1	Random Forest	50587.663161	0.814272

- Did you notice any difference in the execution speed of the two models? Investigate the potential reasons for this difference. Consider factors such as the complexity of the algorithms, their computational requirements, and how they handle data.
- Compare the R^2 scores of the Linear Regression and Random Forest models. What might explain the difference in their performance? Research the characteristics of both algorithms—Linear Regression's simplicity versus Random Forest's ensemble learning approach—and reflect on how they might influence these results.
- Are you curious if other regression models exist that might perform differently? They do! Explore other regression algorithms available in the sklearn library, such as Support Vector Regression (SVR), Gradient Boosting Regression, or K-Nearest Neighbors Regression. Experiment with them and analyze their performance. Be sure to compare the results to deepen your understanding of their strengths and weaknesses.

In [ ]: